Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No: 92410133012**

Aim: To study different word embedding for representation of textual data in vectorized form

IDE: Google Colab

Theory:

Word Embedding is an approach for representing words and documents. Word Embedding or Word Vector is a numeric vector input that represents a word in a lower-dimensional space. It allows words with similar meanings to have a similar representation.

Word Embeddings are a method of extracting features out of text so that we can input those features into a machine learning model to work with text data. They try to preserve syntactical and semantic information. The methods such as Bag of Words (BOW), CountVectorizer and TFIDF rely on the word count in a sentence but do not save any syntactical or semantic information. In these algorithms, the size of the vector is the number of elements in the vocabulary. We can get a sparse matrix if most of the elements are zero. Large input vectors will mean a huge number of weights which will result in high computation required for training. Word Embeddings give a solution to these problems.

Need for Word Embedding?

- To reduce dimensionality
- To use a word to predict the words around it.
- Inter-word semantics must be captured.

How are Word Embeddings used?

- They are used as input to machine learning models. Take the words —-> Give their numeric representation —-> Use in training or inference.
- To represent or visualize any underlying patterns of usage in the corpus that was used to train them.

1. Bag of Words

Bag of Words (BOW) is an algorithm that counts how many times a word appears in a document. Those word counts allow us to compare documents and gauge their similarities for applications like search, document classification, and topic modeling.

2. Tf-Idf

Tf-Idf is shorthand for term frequency-inverse document frequency. So, two things: term frequency and inverse document frequency. Term frequency (TF) is basically the output of the BoW model. For a specific document, it determines how important a word is by looking at how frequently it appears in the document. Term frequency measures the importance of the word. If a word appears a lot of times, then the word must be important. For example, if our document is “I am a cat lover. I have a cat named Steve. I feed a cat outside my room regularly,” we see that the words with the highest frequency are I, a, and cat. This agrees with our intuition that high term frequency = higher importance.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No: 92410133012**

TF(t) = (Number of times term t appears in a document) / (Total number of terms in the document)

IDF used to calculate the weight of rare words across all documents. The words that occur rarely in the corpus have a high IDF score. However, it is known that certain terms, such as "I", "a" may appear a lot of times but have little importance. Thus we need to weigh down the frequent terms while scaling up the rare ones

IDF(t) = log_e(Total number of documents / Number of documents with term t in it)

Pre Lab Exercise:

1. **Perform BoW vectorization of the corpus**

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "I like machine learning",
    "I like deep learning",
    "I enjoy data science"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)

print("Vocabulary:", vectorizer.get_feature_names_out())
    print("BoW Matrix:\n", X.toarray())
```

```
Vocabulary: ['data' 'deep' 'enjoy' 'learning' 'like' 'machine' 'science']

BoW Matrix:

 [[0 0 0 1 1 1 0]
 [0 1 0 1 1 0 0]
 [1 0 1 0 0 0 1]]
```

2. **Perform TF vectorization of the corpus**

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(use_idf=False)
X = vectorizer.fit_transform(corpus)

print("TF Matrix:\n", X.toarray())
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No: 92410133012**

Program (Code):

To be attached with

1. **Perform the TF-IDF vectorization of the corpus**

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "I like machine learning",
    "I like deep learning",
    "I enjoy data science"
]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(corpus)

print("Vocabulary:", vectorizer.get_feature_names_out())
print("TF-IDF Matrix:\n", X.toarray())
```

```
Vocabulary: ['data' 'deep' 'enjoy' 'learning' 'like' 'machine' 'science']

TF-IDF Matrix:

 [[0.         0.         0.         0.51785612 0.51785612 0.68091856
  0.        ]
 [0.         0.68091856 0.         0.51785612 0.51785612 0.
  0.        ]
 [0.57735027 0.         0.57735027 0.         0.         0.
  0.57735027]]
```

code :-

```
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

docs = [
    "I like machine learning",
    "I like deep learning",
    "I enjoy data science"
]

# -------- BoW --------
bow = CountVectorizer()
bow_matrix = bow.fit_transform(docs)
bow_sim = cosine_similarity(bow_matrix)

print("BoW Similarity:\n", bow_sim)
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No: 92410133012**

```
# -------- TF --------
tf = TfidfVectorizer(use_idf=False)
tf_matrix = tf.fit_transform(docs)
tf_sim = cosine_similarity(tf_matrix)

print("\nTF Similarity:\n", tf_sim)

# -------- TF-IDF --------
tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(docs)
tfidf_sim = cosine_similarity(tfidf_matrix)

print("\nTF-IDF Similarity:\n", tfidf_sim)
```

```
BoW Similarity:
 [[1.         0.66666667 0.        ]
 [0.66666667 1.         0.        ]
 [0.         0.         1.        ]]

TF Similarity:
 [[1.         0.66666667 0.        ]
 [0.66666667 1.         0.        ]
 [0.         0.         1.        ]]

TF-IDF Similarity:
 [[1.         0.53634991 0.        ]
 [0.53634991 1.         0.        ]
 [0.         0.         1.        ]]
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No: 92410133012**

Results:

To be attached with

- BoW gives **word count values**
- TF gives **normalized frequency**
- TF-IDF gives **importance of words**
- Common words get low weight
- Rare words get high weight

Observation:

- BoW only counts words, no meaning stored
- TF improves by normalizing frequency
- TF-IDF gives better representation by reducing importance of common words
- TF-IDF is more useful in real NLP tasks
- Output vectors are sparse (many zeros)

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date:**	**Enrolment No: 92410133012**

Post Lab Exercise:

Take three documents and find similarity using

a. **BoW vectorization**
b. **TF Vectorization**
c. **TF-IDF Vectorization**

Comment over the answer obtained in each of the case.

a. BoW Similarity

- Based only on word count
- Gives similarity even if meaning is different
- Less accurate

b. TF Similarity

- Better than BoW
- Considers frequency
- Still not fully meaningful

c. TF-IDF Similarity

- Best among all
- Reduces common words impact
- Captures important words
- Gives more accurate similarity

Final Conclusion

TF-IDF provides better text representation compared to BoW and TF because it considers both frequency and importance of words, resulting in more accurate similarity results.